Project Aether: The Neuro-Symbolic Pivot to Autonomous Reality Architectures

1. Executive Strategy: The Inflection Point

1.1. The Strategic Landscape

The trajectory of artificial intelligence has shifted decisively from the era of **Generative Retrieval**—where systems passively awaited prompts to produce text or images—to the era of **Agentic Autonomy**, characterized by systems that perceive, reason, and act within dynamic environments. Your current operational status, defined by the utilization of a simple video agent, represents a "Legacy AI" pattern: distinct input, processing, and output phases that require human initiation. This model is obsolete in the face of the capabilities offered by your current asset portfolio.

You possess a distinct competitive advantage through a "Micro-Hyperscale" hardware configuration: a flagship consumer GPU (NVIDIA RTX 5080) coupled with a specialized edge inference unit (Intel Core Ultra 7 NPU), backed by the industry's most potent multimodal cloud infrastructure (Google Gemini 3 Pro + Ultra 2.0). The strategic pivot, therefore, is not merely to upgrade software but to fundamentally rearchitect the relationship between your local hardware and the cloud. We must transition from a "Client-Server" model to a **"Split-Brain Neuro-Symbolic" Architecture**, or what we designate as **Project Aether**.

In this architecture, the local hardware ceases to be a mere terminal. The RTX 5080 and Intel NPU become the **"Somatic Nervous System"**—handling immediate perception, reflex actions, and sensory filtering with zero latency and zero marginal cost. The Google Cloud (Vertex AI running Gemini 3 Pro) becomes the **"Cerebral Cortex"**—engaged only for deep reasoning, long-horizon planning, and complex pattern synthesis. This division of labor is not just a technical optimization; it is the only economic and energetic pathway to running a "Revolutionary" agent system that operates 24/7.

1.2. Deconstructing the Asset Portfolio

To architect a revolutionary system, one must first ruthlessly analyze the constraints and capabilities of the available substrate.

The Local Compute Node: The Constraints of Power
The NVIDIA RTX 5080, while a formidable rendering and inference engine, introduces a critical constraint: 16GB of VRAM.1 In the context of Large Language Models (LLMs), this is a "Memory Wall." A state-of-the-art 27B parameter model (like Gemma 2 27B) quantized to

4-bit precision requires approximately 16.5GB of memory simply to load its weights.1 This leaves zero headroom for the Key-Value (KV) Cache—the memory required to store conversation history—or for concurrent visual processing. Consequently, relying on the 5080 for a "Giant Brain" local model is a strategic error that leads to system instability and hallucinations due to aggressive quantization. The strategic imperative for the RTX 5080 is Throughput over Size: running smaller, highly optimized models (Gemma 9B/12B) at extreme speeds to handle the "Fast Loop" of interaction.
The **Intel Core Ultra 7 NPU** represents the most underutilized asset in consumer AI. It is a dedicated Neural Processing Unit designed for asynchronous, low-wattage inference. Its strategic value lies in its ability to function as an **"Always-On Sentinel."** While the GPU draws 300W+ to "listen," the NPU can run Voice Activity Detection (VAD) and continuous speech transcription (Whisper) for milliwatts.[3] This capability unlocks the potential for an agent that never sleeps, constantly monitoring the audio-visual environment for "trigger events" that warrant waking the larger systems.

The Cloud Node: The Infinite Context
Your Google AI Pro Subscription provides access to Gemini Ultra 2.0, but this is primarily through the Gemini Advanced application interface, intended for human-to-bot chat.5 For an autonomous system, we must pivot to the Gemini 3 Pro API via Vertex AI. This model offers a 2-million-token context window, native multimodal understanding (video/audio/code in, text/audio/code out), and the new "Vibe Coding" capability.7 The cloud layer provides the "Infinite Context" that the local GPU lacks, allowing the system to analyze hours of video or entire code repositories in a single inference pass.

1.3. Radical Use Case Concepts

The "Revolutionary" pivot requires applications that are impossible without this specific hybrid architecture. We propose three foundational concepts for Project Aether.

Concept A: The "Infinite Game" Loop (Autonomous Game Architect)

Moving beyond "coding assistants" that paste snippets into an IDE, this agent acts as a **Live Co-Developer and Playtester**.

- **The Workflow:** As you build a game in Unity or Unreal Engine on the RTX 5080, the agent runs in the background. The Intel NPU continuously captures the "Game View" viewport.
- **The Radical Shift:** Utilizing Gemini 3 Pro's **visual reasoning** and **vibe coding** capabilities [7], the agent watches you playtest. If it observes a visual glitch (e.g., "The character is clipping through the floor"), it correlates this visual data with the codebase context it holds in the cloud.
- **The Action:** It does not just alert you; it generates a specific git patch or modifies the Collider parameters in the Unity Inspector via a local script, effectively "debugging reality" in real-time.[10]

Concept B: The "Reality Editor" (Autonomous Director)

This concept transforms the system into a **24/7 Documentary Engine**.

- **The Workflow:** The system ingests continuous video feeds—from security cameras, life-logging wearables, or OBS gameplay streams.
- **The Radical Shift:** Instead of recording terabytes of raw footage, the **Intel NPU** runs a low-power "Interest Filter" (using visual motion and audio intensity models). When a high-entropy event is detected (laughter, an explosion in-game, a person entering a room), the system "wakes" the Gemini 3 Cloud Agent.[4]
- **The Action:** Gemini 3 analyzes the clip, understands the narrative context, generates a voiceover script, synthesizes it using **Gemini Live's native audio** [13], and edits the clip into a "Highlight Reel" automatically. This effectively creates an automated, AI-directed vlog of your life or gameplay.

Concept C: The "Twin" Interface (Neuro-Symbolic Proxy)

This creates a seamless **"Split-Brain" Conversationalist**.

- **The Workflow:** The user interacts with a single chat interface.
- **The Radical Shift:** A local **Gemma 2 9B** model running on the RTX 5080 handles 95% of interactions (opening apps, checking schedules, basic QA) with <20ms latency. The moment a query exceeds its "World Model" (e.g., "Analyze the geopolitical implications of this 50-page PDF"), the local agent detects its own perplexity ceiling and transparently routes the query to **Gemini Ultra 2.0** in the cloud.[14]
- **The Action:** The response returns through the local interface, creating the illusion of a single, omniscient assistant that is both private/fast and infinitely knowledgeable.

---

2. Hardware & Capability Matrix: The Foundation

To execute Project Aether, we must rigorously map capabilities to the hardware components that offer the highest comparative advantage. This prevents the common failure mode of "Cloud-Only" systems (high latency/cost) or "Local-Only" systems (low intelligence/capability).

2.1. Local Compute: The "Reflex" Layer

NVIDIA RTX 5080 (16GB VRAM)

The RTX 5080 is a high-performance graphics card, but its **16GB VRAM** is a hard constraint for AI workloads.

- **The "27B Trap":** There is a prevalent myth that 24GB+ models can run effectively on 16GB cards via aggressive quantization. While technically possible to *load* a **Gemma 2 27B** (Q4_K_M) model (~16.5GB), doing so leaves **zero** VRAM for the KV Cache (context window) or for the GPU's primary job: rendering your desktop or games.[1] Running a 27B model on this card will result in "Context Thrashing" (swapping to system RAM), reducing

token generation speeds to unusable levels (<2 tokens/second).[16]

- **The "Throughput" Strategy:** The strategic move is to utilize the RTX 5080 for **Gemma 2 9B** (or the upcoming Gemma 3 small variants). A 9B model at Q4 precision consumes ~6GB of VRAM. This leaves ~10GB of headroom.[17]
    - **Strategic Value:** This headroom allows for a massive **32k+ token local context window** (essential for remembering conversation history) and enables the concurrent execution of a **Vision Encoder** (like LLaVA or CLIP) to "see" the screen without impacting the desktop experience.
    - **Optimization:** We will employ **NVIDIA TensorRT-LLM**. This library compiles the LLM specifically for the Ada/Blackwell architecture of the 5080, fusing layers and optimizing kernel execution to achieve speeds of 100+ tokens/second, creating a feeling of "instant" intelligence.[18]

Intel Core Ultra 7 NPU (Neural Processing Unit)

The NPU is the key to "Always-On" agency. Unlike the CPU or GPU, which are designed for burst performance, the NPU is designed for sustained, background inference.

- **Architecture:** The Core Ultra NPU functions as a specialized VPU (Vision Processing Unit) optimized for matrix multiplication at low precision (INT8).
- **The "Sentinel" Role:** We will offload the **Whisper (Speech-to-Text)** encoder and **YOLO (Object Detection)** models to the NPU using the **OpenVINO Runtime**.[3]
    - **Whisper offload:** By running the audio encoder on the NPU, the agent can continuously transcribe ambient audio or voice commands without waking the GPU or consuming significant CPU cycles.[21]
    - **Visual Sentinel:** The NPU can process a low-resolution video stream (e.g., 640x480 at 5 FPS) to detect "change" or specific objects. Only when a relevant event is detected does the system trigger the high-power GPU or Cloud pipelines.[4]

2.2. Cloud Compute: The "Reasoning" Layer

Google Cloud Vertex AI (Gemini 3 Pro)

The cloud layer provides the "Deep Thinking" capability.

- **Gemini 3 Pro:** This model is the core of the "Deep Loop." Its **2 million token context window** allows it to ingest entire books, hour-long high-resolution videos, or massive codebases in a single prompt.[23]
- **"Vibe Coding" & Visual Reasoning:** Unlike previous models, Gemini 3 Pro exhibits "agentic" capabilities in coding—it can verify its own code and understand the "vibe" or aesthetic intent of a UI design from a screenshot.[7] This is critical for the Game Architect use case.
- **The "Live" API:** The **Gemini Multimodal Live API** allows for real-time, bidirectional streaming of audio and video via WebSockets. This bypasses the traditional "Record -> Upload -> Process -> Download" latency loop, enabling fluid voice conversations with the

agent.[25]

2.3. The Capability Assignment Matrix

Feature	Local: Intel NPU (Sentinel)	Local: RTX 5080 (Reflex)	Cloud: Gemini 3 Pro (Cortex)
Primary Role	Continuous Monitoring, Wake-Word, Event Filtering.	Immediate Interaction, Rendering, Privacy Proxy.	Deep Reasoning, Complex Coding, Long-Context Analysis.
Model Architecture	Whisper-Small (Encoder), YOLOv8, MobileNet.	Gemma 2 9B (Quantized), LLaVA / Moondream.	Gemini 3 Pro, Gemini Ultra 2.0, Gemini Flash 1.5.
Context Window	Rolling Buffer (10-30 seconds).	8k - 32k Tokens (managed in VRAM).	1 Million - 2 Million Tokens.
Latency	< 10ms (Real-time).	20-50ms (Perceptible).	500ms - 5s (Deliberate).
Power Cost	Negligible (<10W).	Moderate (50W - 300W).	High ($ Cost per Token).
Software Stack	OpenVINO, Intel NPU Acceleration Library.	TensorRT-LLM, Ollama, CUDA.	Vertex AI SDK, Gemini Live API (WebSockets).

---

3. Financial Strategy: The "Free" vs. "Paid" Illusion

A robust architecture must be economically sustainable. The "Revolutionary" agent risks becoming a "Financial Black Hole" if the billing structure is misunderstood.

3.1. The Billing Trap: Subscription vs. API

There is a critical distinction between the **Google One AI Premium** subscription and **Vertex AI** usage.

- **The Subscription ($20/mo):** This grants access to Gemini Advanced in the *web*

interface (gemini.google.com). It allows for human-to-bot chat and integration with Google Docs/Gmail. **It does NOT grant free access to the Gemini API for building custom agents**.[5]
- **The API (Pay-As-You-Go):** Building an autonomous agent requires the **Gemini Developer API** (via AI Studio or Vertex AI). This is billed per 1 million tokens.
  - **The Trap:** While Google AI Studio offers a "Free Tier," it is rate-limited (e.g., 50 requests per day for Pro models) and, crucially, **data submitted to the Free Tier may be used to train Google's models**.[28] For a system processing private video feeds or proprietary game code, you must use the **Paid Tier** (Vertex AI) to ensure data privacy.[29]

3.2. Vertex AI Pricing Dynamics

- **Gemini 3 Pro Costs:**
  - **Input:** $2.00 per 1 million tokens (for prompts < 200k tokens).
  - **Output:** $12.00 per 1 million tokens.
- **Video Tokenization:** Video is tokenized at **263 tokens per second**.[30]
  - **Calculation:** 1 hour of video = 3,600 seconds * 263 tokens/sec = **946,800 tokens**.
  - **Cost:** Analyzing 1 hour of video costs approximately **$1.90** in input fees alone. Running this 24/7 would cost **~$1,368/month**.[31]

3.3. The Economic Savior: Context Caching

To make Project Aether financially viable, we must leverage **Context Caching**.

- **The Mechanism:** Gemini allows you to "cache" large context blocks (e.g., the entire game codebase, the last hour of video, or a massive system instruction set). You pay a one-time fee to upload the cache, and a small hourly storage fee.
- **The Discount:** Subsequent queries against the cached context receive a **~90% discount** on input tokens ($0.20/1M instead of $2.00/1M).[33]
- **Implicit Caching:** Google now supports "Implicit Caching," where if the prefix of your prompt matches a recently seen prompt, it automatically applies the cached rate.[33]
- **Strategic Implementation:**
  - **Game Dev Agent:** Cache the entire Unity project structure and script files (static context). Only the "diff" or the specific bug report is new tokens.
  - **Video Agent:** Cache the "System Instructions" and "Persona" (static). For video, upload chunks to cache, perform multiple analyses (narrative, editing, summary) against the single cache, then discard.

---

4. Technical Architecture: The "Dual-Loop" System

We propose a **Dual-Loop Architecture** that strictly segments tasks based on "Cognitive

Cost" and "Latency Requirements."

4.1. System Topology

The system is constructed as a hierarchy of loops, mimicking biological cognition.

Loop 1: The Somatic Loop (Local / Milliseconds)

- **Hardware:** Intel NPU + RTX 5080.
- **Function:** This loop runs continuously. It handles perception and reflexes.
- **Data Flow:**
  1. **Ingestion:** OpenCV grabs video frames; PyAudio grabs audio.
  2. **Filtering (NPU):** OpenVINO runs VAD and Motion Detection.
  3. **Reflex (GPU):** If a simple command is detected ("Open Unity," "Turn on lights"), **Gemma 2 9B** (via TensorRT-LLM) generates the JSON action payload immediately.
  4. **Action:** Local script executes the command.

Loop 2: The Cognitive Loop (Cloud / Seconds)

- **Hardware:** Vertex AI (Gemini 3 Pro).
- **Function:** This loop runs on demand (escalation). It handles reasoning and creation.
- **Data Flow:**
  1. **Escalation:** The Somatic Loop detects a complex trigger (High entropy visual event, or explicit "Ask Gemini" command).
  2. **Streaming:** The local orchestrator opens a **WebSocket** connection to the **Gemini Live API**.
  3. **Context Transfer:** Audio/Video buffers are streamed to the cloud.
  4. **Reasoning:** Gemini 3 Pro processes the stream, utilizing cached context (e.g., "Game Design Doc").
  5. **Instruction:** Gemini returns a high-level plan (e.g., "Refactor the PlayerController class") or a direct asset (e.g., "Here is the new C# script").
  6. **Execution:** The Somatic Loop receives the plan and executes the file writes or OS commands.

4.2. Memory Architecture: The "Hippocampus"

To bridge the two loops, we need a shared memory state.

- **Vector Database (ChromaDB):** Running locally.
- **Function:** Stores embeddings of past conversations, code snippets, and video summaries.
- **Hybrid Access:**
  - **Gemma (Local):** Queries ChromaDB for "Short Term Memory" (recent user commands).
  - **Gemini (Cloud):** Queries ChromaDB for "Long Term Memory" (project history, user preferences).

- **Synchronization:** When Gemini generates a deep insight, it is summarized by Gemma and stored in ChromaDB, making it accessible to the local loop in the future without re-querying the cloud.

---

5. Software Stack & Implementation Strategy

5.1. The "Hybrid Stack" Definition

Layer	Component	Software Technology
Interface	User UI	**Streamlit** (Web) or **Unity Editor Script** (Game)
Orchestration	Router	**LangChain** (Python) or **LlamaIndex** [15]
Local Inference	LLM Engine	**TensorRT-LLM** (NVIDIA) [18]
Sensory	AI Vision/Audio	**OpenVINO Runtime** (Intel) [3]
Cloud Bridge	API Client	**Google Gen AI SDK** (google-genai) [14]
Communication	Protocol	**WebSockets** (Gemini Live API) [25]
Memory	Vector Store	**ChromaDB** (Local)

5.2. Implementation Guide: The "Vibe Coding" Agent

This details the specific steps to build the **Game Architect** agent.

Step 1: The Sensory Sentinel (NPU)

We use Python with the Intel OpenVINO library to create a lightweight listener.

```
Python
```

```python
# sensory_sentinel.py
import openvino as ov
import numpy as np
#... audio capture libs...

def sensory_loop():
    # Load compiled Whisper Encoder for NPU
    core = ov.Core()
    model = core.read_model("whisper_encoder_npu.xml")
    compiled_model = core.compile_model(model, "NPU")

    while True:
        audio_chunk = capture_audio()
        # VAD Check (Low Power)
        if is_speech(audio_chunk):
            # Transcribe on NPU
            text = run_inference(compiled_model, audio_chunk)
            if "Gemini" in text:
                trigger_cloud_agent(text)
```

Step 2: The Local Reflex (RTX 5080)

We set up a TensorRT-LLM server to handle fast interactions. This avoids the overhead of loading the model for every request.

Bash

```bash
# Launching the Triton Inference Server with TensorRT-LLM backend
docker run --gpus all -v /model_repo:/models nvcr.io/nvidia/tritonserver:24.02-trtllm \
  tritonserver --model-repository=/models
```

Step 3: The Cloud Bridge (Gemini Live API)

This is the core "Revolutionary" component. It streams the screen and audio to Gemini 3.

Python

```python
# cloud_bridge.py
from google import genai
from google.genai import types
import cv2

# Initialize Vertex AI Client
client = genai.Client(vertexai=True, project="my-project", location="us-central1")

async def game_dev_session():
    # Configure for Audio/Video input and Audio/Text output
    config = types.GenerateContentConfig(
        response_modalities=,
        system_instruction="You are a Unity Game Architect. Analyze the video stream for bugs."
    )

    # Establish WebSocket Connection
    async with client.aio.live.connect(model="gemini-3-pro-preview", config=config) as session:
        while True:
            # Capture Unity Viewport
            frame = capture_screen_region(unity_window_rect)

            # Stream to Gemini
            await session.send(input=frame, end_of_turn=False)

            # Receive Analysis
            async for response in session.receive():
                if response.text:
                    print(f"Gemini Suggestion: {response.text}")
                    # If Gemini suggests code, write it to the project
                    if "```csharp" in response.text:
                        write_to_unity_project(response.text)
```

5.3. Managing the 16GB VRAM Limit

To ensure stability on the RTX 5080:

- **VRAM Partitioning:**
  - **OS/Display:** 1.5GB (Reserve).
  - **Gemma 9B (Q4):** 6.0GB (Persistent).
  - **KV Cache:** 4.0GB (Dynamic - allows ~16k context).
  - **Rendering Buffer:** 4.5GB (Leaves room for Unity/Unreal to run).
- **OOM Prevention:** If Unity usage spikes (e.g., baking lighting), the orchestrator must automatically unload the Gemma model to system RAM or pause inference. This logic is

handled by the **Local Router**.

---

6. Radical Use Case Deep Dives

6.1. Deep Dive: The "Reality Editor"

This agent autonomously creates content from your life.

1. **Ingest:** You leave an OBS stream running, capturing your desktop or a webcam.
2. **NPU Filtering:** The local NPU monitors the stream. It calculates an "Entropy Score" based on:
   - **Audio Volume:** Sudden spikes (laughter, shouting).
   - **Visual Motion:** Rapid scene changes.
   - **Semantic Trigger:** Specific keywords detected by Whisper.
3. **Trigger:** When Entropy > Threshold, the NPU marks a "Clip" (start/end timestamps).
4. **Cloud Production:**
   - The orchestrator uploads the Clip to Vertex AI (using **Implicit Caching** if the session is long).
   - **Gemini 3 Pro** is prompted: *"You are a documentary director. Analyze this clip. Write a witty narration script describing the action. Identify the 'climax' of the clip."*
   - **Synthesis:** Gemini 3 Pro returns the script. The system calls the **Gemini Live API** again to synthesize the narration using its high-quality native TTS voices.
   - **Edit:** A local **FFmpeg** script (generated by Gemini) cuts the video, overlays the narration audio, adds subtitles (from the transcription), and saves the file as Highlight_YYYYMMDD.mp4.
5. **Result:** You wake up to a folder of perfectly edited highlights from your previous day's work or gaming.

6.2. Deep Dive: The "Infinite Game"

1. **Context Loading:** The agent recursively reads your Assets/Scripts folder in Unity. It concatenates the C# files into a single text block and uploads it to Vertex AI with **Context Caching** enabled (TTL: 24 hours).
2. **Visual Debugging:** You are playtesting. The player character gets stuck on a wall.
3. **Intervention:** You ask, *"Gemini, fix this collision bug."*
4. **Reasoning:**
   - Gemini 3 Pro receives the video frames of the character getting stuck.
   - It cross-references the visual evidence with the cached PlayerController.cs and Wall.cs scripts.
   - It identifies that OnCollisionEnter has a logic error regarding the wall's tag.
5. **Execution:** Gemini generates the corrected function. The local agent overwrites the file. Unity automatically recompiles. You try again, and the bug is gone.

7. Future Outlook: The "Centaur" Paradigm

The architecture defined in **Project Aether** represents the future of high-end consumer AI. It rejects the binary choice between "Dumb Local" and "Expensive Cloud." Instead, it creates a **Centaur**—a system that is functionally essentially human-like in its ability to be both reflexive and reflective.

By leveraging the **RTX 5080** for the body (immediate action, rendering, filtering) and **Gemini 3 Pro** for the mind (reasoning, creativity, coding), you bypass the limitations of both. You avoid the bankruptcy of pure cloud reliance via NPU filtering and Context Caching. You avoid the stupidity of pure local reliance via the "Infinite Context" of the cloud.

Final Recommendation:
Begin by constructing the Sensory Sentinel on your NPU. This is the "Spark of Life" for the system. Once your computer can "hear" and "see" without burning 300 Watts, the rest of the architecture—the routing, the reasoning, the editing—will follow as a natural evolution of this new, autonomous nervous system. The tools (OpenVINO, TensorRT, Vertex AI) are ready; the integration is the revolution.

Works cited

1. Could 16gb vram be enough for the 27b QAT version? - Hacker News, accessed January 16, 2026, https://news.ycombinator.com/item?id=43743631
2. How I Run Gemma 3 27B on an RX 7800 XT 16GB Locally! : r/LocalLLaMA - Reddit, accessed January 16, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1kjwi3w/how_i_run_gemma_3_27b_on_an_rx_7800_xt_16gb/
3. Automatic speech recognition using Whisper and OpenVINO with Generate API, accessed January 16, 2026, https://docs.openvino.ai/2024/notebooks/whisper-asr-genai-with-output.html
4. Show HN: Quick and dirty speech transcription on an Intel NPU - Hacker News, accessed January 16, 2026, https://news.ycombinator.com/item?id=42347114
5. Im paying for Ultra and Im being told Im out of free credits when building my app with gemini., accessed January 16, 2026, https://support.google.com/gemini/thread/361038522/im-paying-for-ultra-and-im-being-told-im-out-of-free-credits-when-building-my-app-with-gemini?hl=en
6. Google Gemini Pricing Guide: What You Need to Know, accessed January 16, 2026, https://www.cloudeagle.ai/blogs/blogs-google-gemini-pricing-guide
7. Gemini 3 Pro - Google DeepMind, accessed January 16, 2026, https://deepmind.google/models/gemini/pro/
8. Vibe coding with Gemini 3 in AI Studio, accessed January 16, 2026, https://www.youtube.com/watch?v=Pb6XHGi542A
9. How I “Vibe Coded” the Future of News with Gemini 3 | by Dimitre Oliveira | Data

Science Collective | Dec, 2025, accessed January 16, 2026, https://medium.com/data-science-collective/how-i-vibe-coded-the-future-of-news-with-gemini-3-65955add1167
10. Game Development with Gemini 3 - Medium, accessed January 16, 2026, https://medium.com/@jengas/game-development-with-gemini-3-01cdcc226553
11. Real-World Agent Examples with Gemini 3 - Google for Developers Blog, accessed January 16, 2026, https://developers.googleblog.com/real-world-agent-examples-with-gemini-3/
12. Intel Core Ultra & Windows Copilot: What local AI can (and can't) do | PCWorld, accessed January 16, 2026, https://www.pcworld.com/article/2906601/intel-core-ultra-windows-copilot-which-local-ai-functions-can-you-control.html
13. Multimodal Live API - Quickstart - Colab - Google, accessed January 16, 2026, https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_LiveAPI.ipynb
14. Google Gen AI SDK documentation - googleapis.github.io, accessed January 16, 2026, https://googleapis.github.io/python-genai/
15. Agent Workflows | Control flow for your GenAI apps - LlamaIndex, accessed January 16, 2026, https://www.llamaindex.ai/workflows
16. Unusually high VRAM usage of Gemma 3 27B · Issue #9678 - GitHub, accessed January 16, 2026, https://github.com/ollama/ollama/issues/9678
17. Best Local LLMs for Every NVIDIA RTX 40 Series GPU - ApX Machine Learning, accessed January 16, 2026, https://apxml.com/posts/best-local-llm-rtx-40-gpu
18. NVIDIA TensorRT-LLM Revs Up Inference for Google Gemma | NVIDIA Technical Blog, accessed January 16, 2026, https://developer.nvidia.com/blog/nvidia-tensorrt-llm-revs-up-inference-for-google-gemma/
19. Building a TensorRT-LLM Engine - NVIDIA Documentation, accessed January 16, 2026, https://docs.nvidia.com/deeplearning/tensorrt-cloud/v0.5.3-ea/user/build-trt-llm-engine.html
20. OpenVINO GenAI on NPU, accessed January 16, 2026, https://docs.openvino.ai/2025/openvino-workflow-generative/inference-with-genai/inference-with-genai-on-npu.html
21. Slow performance on Intel NPU · Issue #364 · intel/openvino-plugins-ai-audacity - GitHub, accessed January 16, 2026, https://github.com/intel/openvino-plugins-ai-audacity/issues/364
22. Automatic Speech Recognition Using OpenAI* Whisper without a GPU - Intel, accessed January 16, 2026, https://www.intel.com/content/www/us/en/developer/articles/technical/speech-recognition-in-openai-whisper-without-a-gpu.html
23. Gemini 1.5 user inputs 350k token Mr. Beast video and model incorporates minor details from various parts of the video to successfully answer complex question. : r/singularity - Reddit, accessed January 16, 2026, https://www.reddit.com/r/singularity/comments/1awg9c9/gemini_15_user_inputs_3

50k_token_mr_beast_video/
24. Gemini 3 Developer Guide | Gemini API - Google AI for Developers, accessed January 16, 2026, https://ai.google.dev/gemini-api/docs/gemini-3
25. Get started with the Gemini Live API using Firebase AI Logic - Google, accessed January 16, 2026, https://firebase.google.com/docs/ai-logic/live-api
26. generative-ai/gemini/multimodal-live-api/intro_multimodal_live_api_genai_sdk.ipynb at main - GitHub, accessed January 16, 2026, https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/multimodal-live-api/intro_multimodal_live_api_genai_sdk.ipynb
27. Why Gemini Advance subscription doesn't come with an API plan? : r/Bard - Reddit, accessed January 16, 2026, https://www.reddit.com/r/Bard/comments/1jplho0/why_gemini_advance_subscription_doesnt_come_with/
28. Google Developer Program Benefits FAQ, accessed January 16, 2026, https://developers.google.com/profile/help/benefits
29. What is the difference between AI Studio and Vertex ? : r/Bard - Reddit, accessed January 16, 2026, https://www.reddit.com/r/Bard/comments/1oq4tfb/what_is_the_difference_between_ai_studio_and/
30. Understand and count tokens | Gemini API | Google AI for Developers, accessed January 16, 2026, https://ai.google.dev/gemini-api/docs/tokens
31. How Much Are Gemini 3 Pro and Gemini 3 API in 2026 - GlobalGPT, accessed January 16, 2026, https://www.glbgpt.com/hub/gemini-3-pro-costs-gemini-3-api-costs-latest-insights-for-2025/
32. Vertex AI Pricing | Google Cloud, accessed January 16, 2026, https://cloud.google.com/vertex-ai/generative-ai/pricing
33. Context caching overview | Generative AI on Vertex AI - Google Cloud Documentation, accessed January 16, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview
34. Vertex AI context caching | Google Cloud Blog, accessed January 16, 2026, https://cloud.google.com/blog/products/ai-machine-learning/vertex-ai-context-caching
35. Gemini 2.5 Models now support implicit caching - Google Developers Blog, accessed January 16, 2026, https://developers.googleblog.com/en/gemini-2-5-models-now-support-implicit-caching/
36. Leverage LlamaIndex with Vertex AI Vector Search to perform question answering RAG - GitHub, accessed January 16, 2026, https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/use-cases/retrieval-augmented-generation/llamaindex_rag.ipynb